

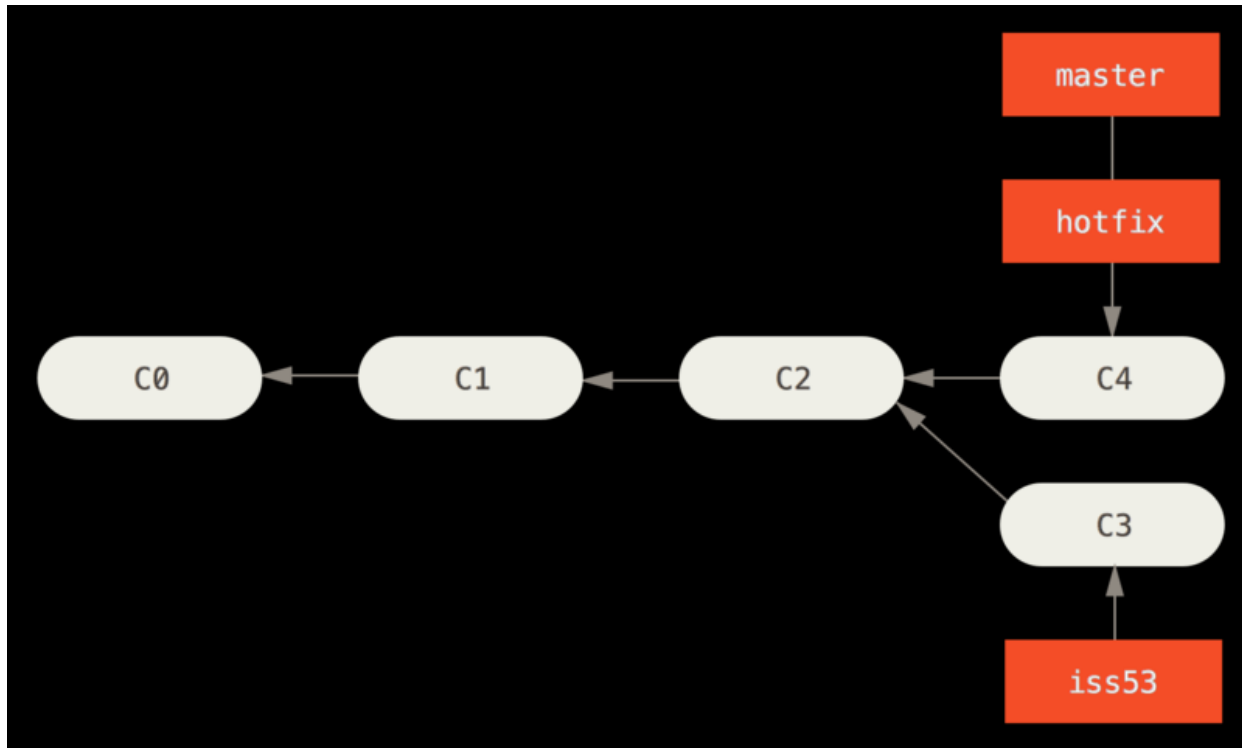
Git 合并与变基

Merge

Fast-forward Merge , hotfix 的直接上游分支(master)就是 C2

`git checkout master // (@ C2)`

`git merge hotfix // after that, (HEAD / master) --> C4`



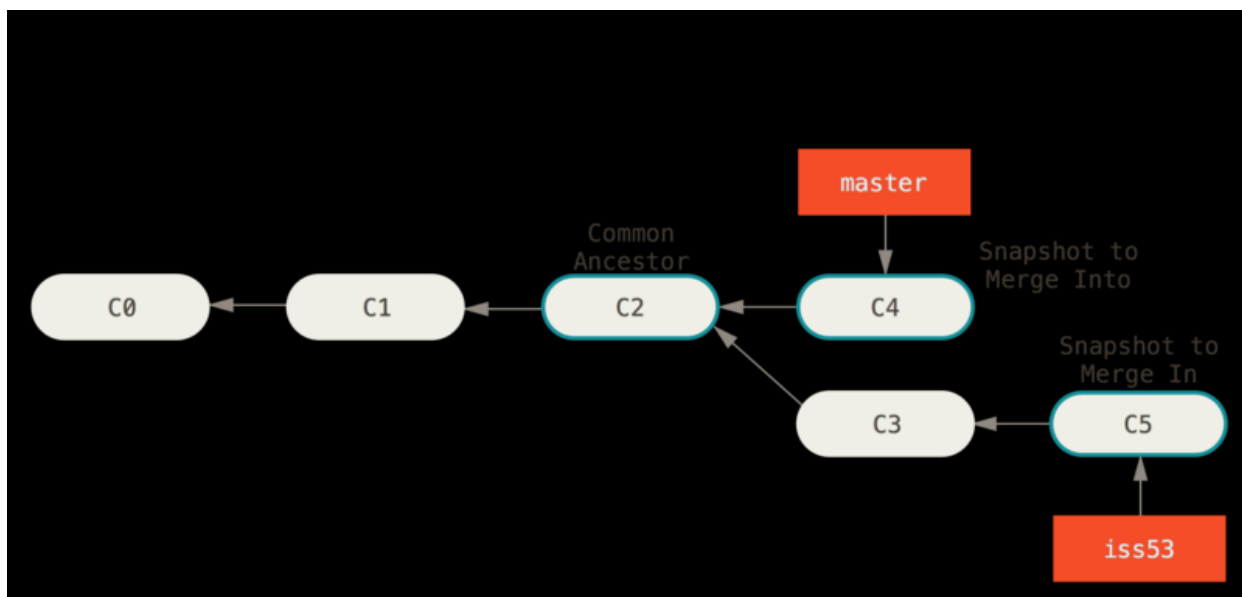
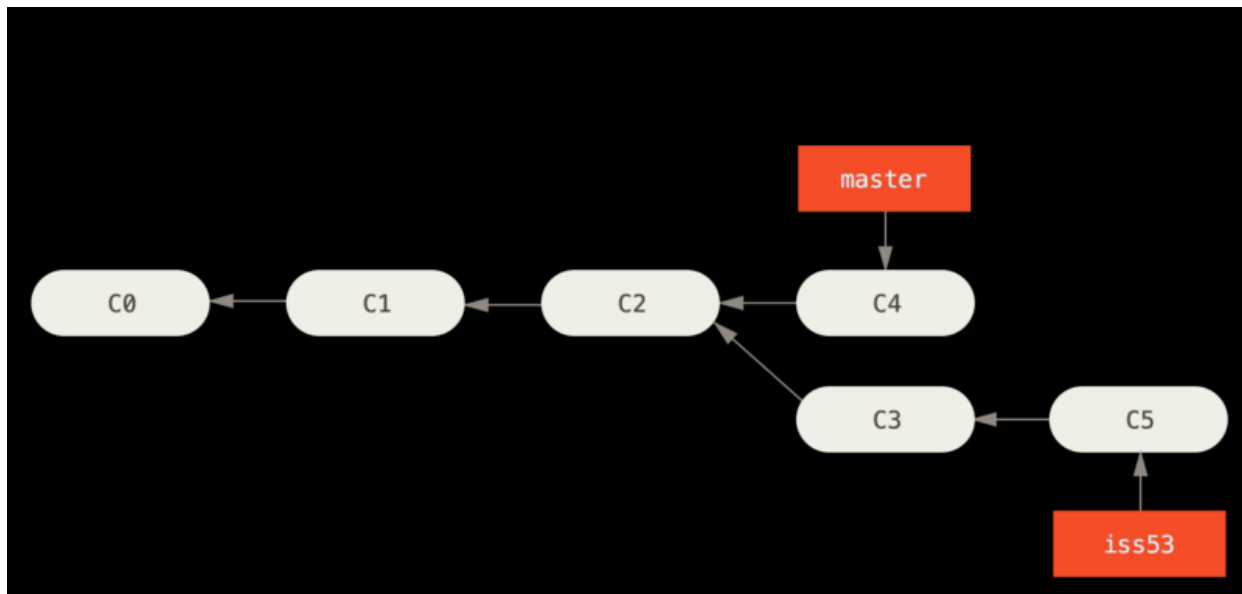
Recursive Merge

`git checkout master // (@ C4)`

`git merge iss53 //`

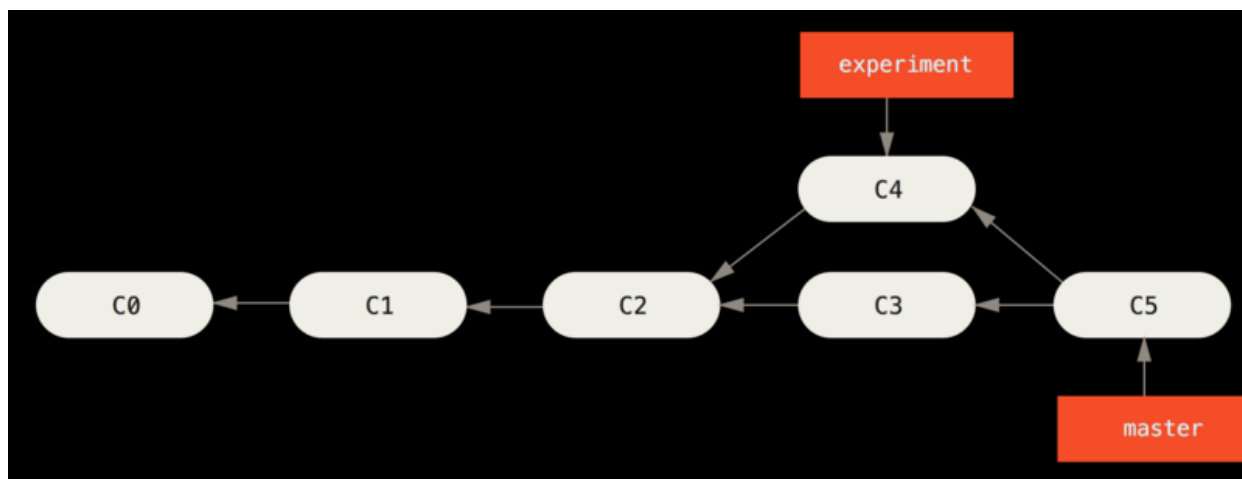
三方合并

这和你之前合并 hotfix 分支的时候看起来有一点不一样。在这种情况下，你的开发历史从一个更早的地方开始分叉开来 (diverged)。因为，master 分支所在提交并不是 iss53 分支所在提交的直接祖先，Git 不得不做一些额外的工作。出现这种情况的时候，Git 会使用两个分支的末端所指的快照 (C4 和 C5) 以及这两个分支的共同的祖先 (C2)，做一个简单的三方合并



之前介绍过，整合分支最容易的方法是 `merge` 命令。它会把两个分支的最新快照（C3 和 C4）以及二者最近共同祖先（C2）进行三方合并，合并的结果是生成一个新的快照（并提交）

Rebase



当前的master 指向C3, 执行完以下命令就指向了C5

```
$ git checkout experiment $ git rebase master First, rewinding head to replay your work on top of it... Applying: added staged command
```

其实，还有一种方法：**你可以提取在 C4 中引入的补丁和修改，然后在 C3 的基础上应用一次**。在 Git 中，这种操作就叫做 **变基**。你可以使用 `rebase` 命令将提交到某一支上的所有修改都移至另一分支上，就好像“重新播放”一样。

它的原理是首先找到这两个分支（即当前分支 `experiment`、变基操作的目标基底分支 `master`）的最近共同祖先 `C2`，然后对比当前分支相对于该祖先的历次提交，提取相应的修改并存储为临时文件，然后将当前分支指向目标基底 `C3`，最后以此将之前另存为临时文件的修改依序应用。（译注：写明了 commit id，以便理解，下同）

具体的，就是提取 C4 相比较于C2的修改差异，然后将这个差异应用到C3(master分支当前的指针指向C3)上,然后将这个差异，应用于C3

因为，如果rebase 成功，那么历史记录应该是

```
$ git checkout master
```

```
$ git log
```

C3 : msg 3 <== master 分支现在的HEAD指向 C3

C2 : msg 2

C1 : msg 1

C0 : msg 0

```
$ git checkout experiment
```

```
$ git log
```

C4 : msg 4 <== experiment 分支现在的HEAD指向 C4

C2 : msg 2

C1 : msg 1

C0 : msg 0

//

// rebase 基础的原理

// 在C3的基础上，应用 C2 --> C4 的差异

//

\$ git rebase master

First, rewinding head to replay your work on top of it... Applying: added staged command

\$ git log

C4 : msg 4 <== experiment 分支现在的HEAD指向 C4

C3 : msg 3 <== master 分支现在的HEAD指向 C3

C2 : msg 2

C1 : msg 1

C0 : msg 0

\$ git checkout master

\$ git log

C3 : msg 3 <== master 分支现在的HEAD指向 C3

C2 : msg 2

C1 : msg 1

C0 : msg 0

\$ git merge experiment // Fast-forward merge

\$ git log

C4 : msg 4 <== experiment 分支现在的HEAD指向 C4

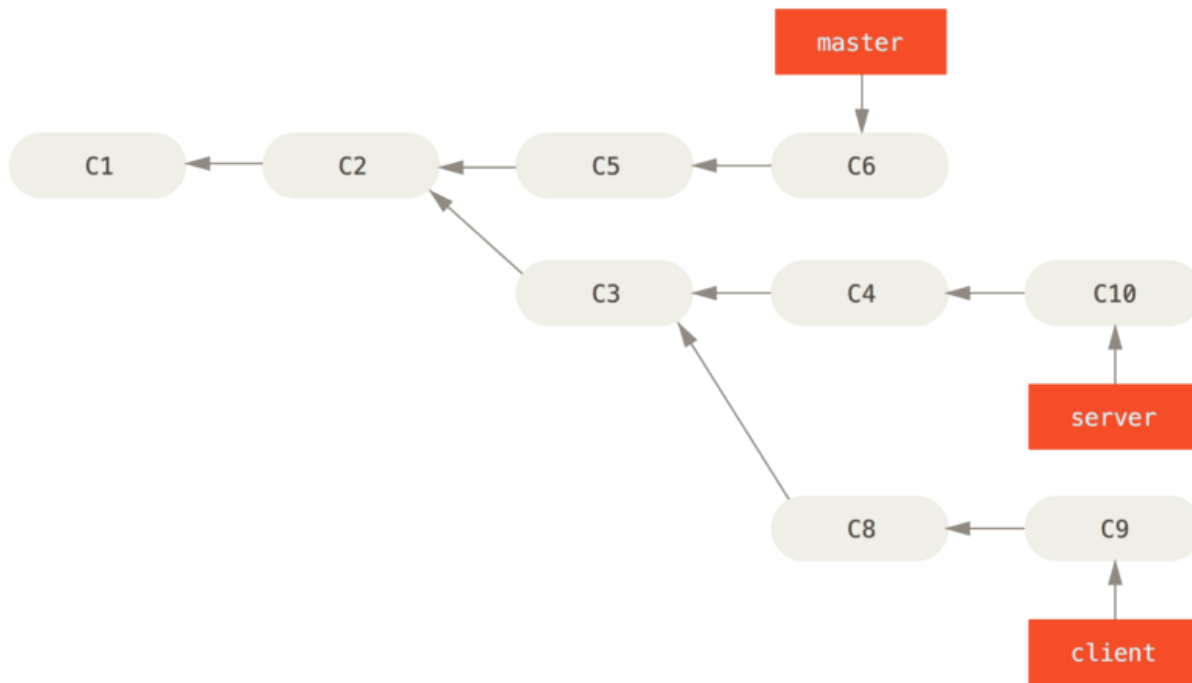
C3 : msg 3 <== master 分支现在的HEAD指向 C3

C2 : msg 2

C1 : msg 1

C0 : msg 0

更有趣的变基例子



// 在server分支上，执行如下命令，将在client的修改，但是不在server上的修改变基到 master上合并应用到 client分支，注意执行完以后命令后，**从当前分支从 server 分支，切回到了 client分支**，并已经将属于 client分支的改动，但不属于server分支的改动，变基合并到了master分支了了
\$ git rebase --onto master server client **// 执行完后，你处于 Client分支**

// 执行前是在 master分支(但已经合并入了client分支的修改的内容)

// 执行后是在 server分支(但已经合并入了master分支的修改的内容 (即Client分支的修改))

// 再把master分支上的改动，变基到server分支

接下来你决定将 server 分支中的修改也整合进来。使用 `git rebase [basebranch] [topicbranch]` 命令可以直接将特性分支（即本例中的 server）变基到目标分支（即 master）上。这样做能省去你先切换到 server 分支，再对其执行变基命令的多个步骤。

\$ git rebase master server

git 内部原理

// 将 test content 这段文本，写入 .git/objects 目录下

\$ echo 'test content' | git hash-object -w --stdin

d670460b4b4aece5915caf5c68d12f560a9fe3e4

```
$ find .git/objects -type f .git/objects/d6/70460b4b4aece5915caf5c68d12f560a9fe3e4
```

```
$ git cat-file -p d670460b4b4aece5915caf5c68d12f560a9fe3e4 test content
```

```
$ git cat-file -t 1f7a7a472abf3dd9643fd615f6da379c4acb3e3a blob
```

// 可以查看 `git reset --hard <commit-id>` 之前的那一次 `<commit-id>`

// 也可以用 `git reflog` 来查看

```
$ git log -g
```